

JjTL and JjEL: A Set-Theoretic Approach to Model Transformation and Expression Languages

Jjodel Project: Language Design Document

March 9, 2026

Abstract

This document presents the design and rationale of two domain-specific languages developed within the Jjodel ecosystem: JjTL (Jjodel Transformation Language), a declarative Model-to-Model transformation language, and JjEL (Jjodel Expression Language), a set-theoretic expression language inspired by first-order logic. We describe the syntax, semantics, and, most importantly, the *design motivations* behind each language construct, explaining *why* certain decisions were made and what alternatives were considered. We then provide a detailed comparison with four established transformation languages: ATL, EOL/ETL, QVT Relations, and QVT Operational Mappings. Finally, we present a series of parallel transformation examples across ATL, ETL, and JjTL to enable direct syntactic and conceptual comparison.

Contents

1	Introduction	4
1.1	Design Criteria	4
1.2	Document Structure	4
2	JjEL: Jjodel Expression Language	5
2.1	Positioning and Scope	5
2.2	Types and Values	5
2.3	The Core Construct: <code>forall</code>	5
2.3.1	Design Motivation	5
2.3.2	Syntax and Forms	6
2.3.3	Separator Design	6
2.3.4	Why Not <code>filter</code> and <code>map</code> ?	6
2.4	Existential Check: <code>exists</code>	6
2.5	Logical Implication: <code>implies</code>	7
2.6	Context Binding: <code>with...do</code>	7
2.7	Conditional: <code>if...then...else</code>	7
2.8	Lambda Expressions	7
2.9	Null Safety	8
2.10	Operator Precedence	8
2.11	Built-in Methods	8
2.12	Evaluation Rules	9
2.12.1	Short-Circuit Evaluation	9
2.12.2	Truthiness	9
2.12.3	Error Handling	9
3	JjTL: Jjodel Transformation Language	9
3.1	Design Philosophy	9
3.2	Transformation Structure	9
3.3	Class Mappings	10
3.3.1	Multiplicity	10

3.3.2	Guard Conditions	10
3.4	Attribute Mappings	10
3.4.1	Direct Mapping	10
3.4.2	Value Mapping (Enum Conversion)	11
3.4.3	Expression Mapping	11
3.4.4	Object Creation	11
3.5	Helper Functions	11
3.6	Trace Model	11
3.6.1	Invertibility Analysis	11
3.7	Interactive Features	12
3.8	Execution Model	12
4	Comparative Analysis	12
4.1	ATL (Atlas Transformation Language)	12
4.1.1	Overview	12
4.1.2	Structural Comparison	13
4.1.3	Key Design Differences	13
4.2	EOL / ETL (Epsilon Platform)	13
4.2.1	Overview	13
4.2.2	Structural Comparison	14
4.2.3	Key Design Differences	14
4.3	QVT Relations (QVT-R)	14
4.3.1	Overview	14
4.3.2	Structural Comparison	14
4.3.3	Key Design Differences	15
4.4	QVT Operational (QVT-O)	15
4.4.1	Overview	15
4.4.2	Structural Comparison	15
4.4.3	Key Design Differences	15
4.5	Feature Comparison Matrix	16
4.6	Expression Language Comparison	16
5	Transformation Examples	16
5.1	Example 1: UML Class to Relational Table	17
5.1.1	ATL	17
5.1.2	ETL	18
5.1.3	JjTL	18
5.2	Example 2: State Machine to Petri Net	19
5.2.1	ATL	19
5.2.2	ETL	20
5.2.3	JjTL	20
5.3	Example 3: UML to Java (Structural)	21
5.3.1	ATL	21
5.3.2	ETL	22
5.3.3	JjTL	23
5.4	Example 4: Validation Invariants	23
5.5	Example 5: Guard Conditions	24
6	Discussion	24
6.1	Strengths of the JjTL/JjEL Approach	24
6.2	Limitations and Trade-offs	25
6.3	Open Design Points	25

7 Conclusion	25
A JjEL Grammar (Complete)	26
B JjEL Built-in Methods (Complete)	27
B.1 String Methods (33)	27
B.2 Collection Methods (28)	27
C JjTL Token Types	29

1 Introduction

Model-Driven Engineering (MDE) relies on model transformations as a fundamental mechanism for bridging abstraction levels, synchronizing representations, and generating artifacts from high-level specifications. Over the past two decades, a rich ecosystem of transformation languages has emerged, from the OMG’s standardized QVT family to community-driven tools like ATL and Epsilon. Despite this maturity, adoption of dedicated transformation languages in industrial practice remains limited, with many practitioners preferring general-purpose languages like Java or Kotlin.

We identify three root causes for this adoption gap:

- (i) **Cognitive overhead:** Existing languages require learning OCL, understanding complex execution semantics (e.g., ATL’s two-phase execution, QVT-R’s bidirectional pattern matching), and dealing with Eclipse-specific tooling.
- (ii) **Syntax-semantics mismatch:** Languages like QVT-R use declarative syntax but require procedural reasoning about enforcement behavior; ATL mixes declarative bindings with imperative `do` blocks under different execution models.
- (iii) **Expression language limitations:** OCL, while mathematically rigorous, was designed for *constraint specification* (what must be true), not for *value computation* (which elements, which values), yet it serves as the expression sub-language for ATL, QVT-R, and QVT-O.

The Jjodel project addresses these issues with two complementary languages:

- **JjEL**, a set-theoretic expression language whose core construct, `forall`, has *set comprehension* semantics (it returns a set, not a boolean). This bridges the gap between formal logic and practical computation, providing a single primitive that replaces `filter`, `map`, and logical quantification.
- **JjTL**, a declarative M2M transformation language that uses JjEL as its expression sub-language, offering a simpler rule structure than ATL while maintaining automatic traceability and guard conditions.

1.1 Design Criteria

Both languages were designed under three guiding principles, applied in strict priority order:

1. **Maximum declarativity.** The user describes *what* they want, never *how* to compute it. No assignment, no loops, no mutation.
2. **Minimum cognitive load.** Few rules to remember, minimal syntax, no boilerplate. The most frequent MDE operations are available as native methods.
3. **Maximum intuitiveness.** Constructs should read almost like natural language. Textual operators (`and`, `or`, `not`, `implies`), first-order-logic-inspired quantifiers (`forall`, `exists`), and null-safe navigation (`?.`, `??`).

1.2 Document Structure

Section 2 presents JjEL in detail, including its set-theoretic semantics and design motivations. Section 3 describes JjTL and its transformation model. Section 4 provides a detailed comparative analysis with ATL, EOL/ETL, QVT-R, and QVT-O. Section 5 presents parallel transformation examples across multiple languages. Section 6 discusses open points and future directions.

2 JjEL: Jjodel Expression Language

JjEL is a pure, side-effect-free expression language designed for navigating, querying, and computing values over model graphs. Every JjEL expression produces a value; there are no statements.

2.1 Positioning and Scope

JjEL occupies a specific niche in the MDE language landscape:

OCL	JjEL	EOL	JavaScript
Formal constraints “what must be true”	Set-theoretic expressions “which elements, which values”	Model scripting “manipulate the model”	General-purpose “computation”

JjEL is *not* a constraint language (it computes values, not assertions), not a programming language (no variables, loops, or I/O), and not a transformation language (it provides expressions *inside* rules, not rules themselves). JjEL provides the expression sub-language for JjTL, analogous to how OCL provides expressions for ATL and QVT.

The scope boundary is precise: JjTL handles Model-to-Model (M2M) transformations; Model-to-Text (M2T) generation is handled by a separate template engine.

2.2 Types and Values

JjEL supports five primitive types and three composite types:

Category	Type	Literals	Examples
Primitive	EString	Double-quoted	"hello", "Customer"
	EInt	Integer	0, 42, -7
	EDouble	Decimal	3.14, .5
	EBoolean	Keyword	true, false
	null	Keyword	null
Composite	Array	Bracket	[], [1, 2, 3]
	Object	(no literal)	Model elements
	Function	Lambda	a => a.name

The type system includes aliases for interoperability: EString/String, EInt/Integer, EDouble/Double/Number, EBoolean/Boolean. The `is` operator checks type membership including supertypes (kind-of semantics).

2.3 The Core Construct: forall

The most important design decision in JjEL is the semantics of `forall`.

2.3.1 Design Motivation

In classical first-order logic, the universal quantifier $\forall$ is an *assertion*: $\forall x \in S : P(x)$ returns a boolean (true if P holds for every x). In set theory, the set-builder notation $\{x \in S \mid P(x)\}$ *selects* elements, returning a set.

We chose **set-theoretic semantics**: `forall` returns a *set*, not a boolean. The rationale:

1. **Computational utility**: In model transformations, we almost always need to *select* elements and *compute* values from them. A set-returning construct is directly useful; a boolean-returning one requires additional machinery.

2. **Unification:** With set-theoretic `forall`, the `filter` and `map` collection methods become redundant; `forall` subsumes both. This reduces the number of concepts a user must learn.
3. **Natural reading:** “for all attributes such that they are public: get their name” reads naturally and maps directly to the syntax.

2.3.2 Syntax and Forms

`forall` has four forms, distinguished by three separators with distinct roles:

```
forall <var> in <collection> [such that <condition>] [: <projection>] [do <action>]
```

Form	Returns	Equivalent to	Example
<code>forall x in S such that P</code>	Set	<code>S.filter(P)</code>	<code>forall a in attrs such that</code>
<code>forall x in S: expr</code>	Set	<code>S.map(expr)</code>	<code>forall a in attrs: a.name</code>
<code>forall x in S such that P: expr</code>	Set	<code>S.filter(P).map(expr)</code>	<code>forall a in attrs such that</code>
<code>forall x in S do action</code>	–	Imperative loop	<code>forall a in attrs do create</code>

2.3.3 Separator Design

The three separators are *not* interchangeable; each has a precise role:

- `such that` introduces a *boolean filter condition*. It reads as a natural qualifier: “all attributes *such that* they are public.”
- `:` introduces a *value projection*. It reads as “give me”: “all attributes: their name” (i.e., the names of all attributes).
- `do` introduces an *imperative action*. It is reserved for JjTL transformation actions and is not available in pure JjEL expressions. This separation ensures JjEL remains side-effect-free.

The decision to use `such that` (two words) rather than a symbol like `|` was deliberate: it prioritizes readability and matches the set-builder notation commonly taught in discrete mathematics courses ($\{x \in S \mid P(x)\}$ is read “ x in S *such that* $P(x)$ ”).

2.3.4 Why Not filter and map?

The `filter/map` pair requires the user to:

1. Know two distinct method names and their order (filter first, then map).
2. Write a lambda for each: `attrs.filter(a => a.isPublic).map(a => a.name)`.
3. Understand method chaining and intermediate collections.

With `forall`, the same expression becomes:

```
forall a in attrs such that a.isPublic: a.name
```

This is one concept instead of three, with a single binding variable (`a`) instead of two lambdas. The cognitive load reduction is significant, especially for domain experts who are not programmers.

2.4 Existential Check: exists

While `forall` returns a set, `exists` returns a **boolean**: does at least one element satisfy the condition?

```
exists a in attributes: a.type == "String"
exists a in attributes such that a.isPublic and not a.isDerived
```

`exists` is semantically derived from `forall`:

$$\text{exists } x \text{ in } S: P \equiv (\text{forall } x \text{ in } S \text{ such that } P). \text{isEmpty}$$

For `exists`, the separators `:` and `such that` are *synonyms*: both introduce the boolean predicate. This is because `exists` has no projection: it always returns boolean. The synonym design avoids forcing users to remember which separator to use for `exists`.

2.5 Logical Implication: `implies`

```
isAbstract implies subClasses.isEmpty
a.isPublic implies a.type != null
```

`implies` is semantically equivalent to `not P or Q` but reads as natural language. It is right-associative, short-circuit evaluated, and has lower precedence than `and/or`.

The motivation is direct: model validation invariants frequently express implications (“if a class is abstract, then it must have subclasses”). Without `implies`, users must negate the antecedent manually (`not isAbstract or subClasses.isEmpty`), which is less readable and more error-prone.

2.6 Context Binding: `with...do`

```
with selectedNode do name.pascalCase()
with selectedNode do forall a in attributes such that a.isPublic: a.name
```

The `with` construct establishes a context object whose properties become directly accessible in the body. This is essential in the Console REPL, where there is no implicit source element (unlike JjTL rules where the matched element provides implicit context).

The choice of `do` as the body separator (rather than a block or colon) was deliberate: it reads naturally (“with this node, *do* [evaluate] this expression”) and parallels the imperative `do` in `forall` by indicating “what to compute in this context.”

2.7 Conditional: `if...then...else`

```
if isAbstract then "abstract" else "concrete"
if lowerBound == 0 then "NULL" else "NOT NULL"
```

This construct is retained from common expression languages. An alternative syntax (`when...then...otherwise`) was considered for its more declarative flavor, but `if/then/else` was kept for its universality and zero learning curve.

When `else` is omitted, the expression returns `null` for the false branch, maintaining the invariant that every expression produces a value.

2.8 Lambda Expressions

```
a => a.name
(a, b) => a + b
```

Lambdas use the `=>` arrow syntax. An earlier design used `:` as the lambda separator (`a: a.name`), but this conflicted with the projection role of `:` in `forall`. The switch to `=>` freed `:` for its set-theoretic role and made the grammar LL(1)-parseable without backtracking.

Lambdas are primarily used with collection methods not subsumed by `forall`: `sortBy`, `groupBy`, `distinctBy`, etc.

2.9 Null Safety

```
parent?.name           -- null if parent is null
parent?.name ?? "no parent" -- "no parent" if chain is null
element?.container?.package?.name ?? "default"
```

Two operators handle null propagation:

- `?.` (null-safe navigation): returns `null` if the receiver is null, without throwing.
- `??` (null coalesce): returns the right operand if the left is null.

Using `.` on a null value throws an error by design: this forces users to explicitly acknowledge potential nulls via `?.`, making null handling visible rather than silently propagating.

2.10 Operator Precedence

Level	Construct	Associativity
1 (lowest)	<code>if ... then ... else ...</code>	right
2	<code>??</code> (null coalesce)	left
3	<code>implies</code>	right
4	<code>or</code>	left
5	<code>and</code>	left
6	<code>==, !=</code>	left
7	<code><, >, <=, >=</code>	left
8	<code>is</code>	—
9	<code>+, -</code>	left
10	<code>*, /, %</code>	left
11	<code>not, -</code> (unary)	right
12 (highest)	<code>., ?., [index]</code>	left

2.11 Built-in Methods

JjEL provides over 125 built-in methods organized by type. The guiding principle is that the most frequent MDE operations should be native, requiring no import or boilerplate.

Category	Count	Highlights
String	33	<code>camelCase()</code> , <code>pascalCase()</code> , <code>snakeCase()</code> , <code>kebabCase()</code>
Number	30	<code>abs()</code> , <code>clamp()</code> , <code>between()</code> , trigonometry
Collection	28	<code>sortBy()</code> , <code>groupBy()</code> , <code>flatten()</code> , <code>distinct()</code>
Date	39	<code>addDays()</code> , <code>diffMonths()</code> , <code>format()</code>

Naming convention methods (`camelCase`, `pascalCase`, `snakeCase`, `kebabCase`) are native because they are essential in model transformations (e.g., converting class names to table names, attribute names to column names). No other transformation language provides these natively.

Note: `filter()` and `map()` are *removed* from the collection methods, as they are replaced by the `forall` construct. The remaining collection methods are those that `forall` does not subsume: aggregation (`sum`, `avg`), ordering (`sortBy`), structure (`groupBy`, `flatten`), and boolean checks (`all`, `any`, `none`).

2.12 Evaluation Rules

2.12.1 Short-Circuit Evaluation

`and` and `or` use **short-circuit** evaluation: the right operand is not evaluated if the result is determined by the left operand. This is essential for guard-like expressions:

```
parent != null and parent.name.isNotEmpty    -- safe: parent.name not reached if null
```

2.12.2 Truthiness

JjEL uses a broad truthiness model: `null`, `false`, `0`, `""` (empty string), and `[]` (empty array) are all falsy. This enables concise guards (`if attributes then ...` instead of `if attributes.isNotEmpty then ...`).

2.12.3 Error Handling

JjEL distinguishes between recoverable and non-recoverable errors:

- **Silent null:** undefined variables, property not found on object (with console warning and typo suggestions via Levenshtein distance), division by zero, type mismatches in arithmetic.
- **Thrown error:** `.property` on `null` (forces use of `?.`), unknown method call (with suggestions).

3 JjTL: Jjodel Transformation Language

JjTL is a declarative, rule-based Model-to-Model transformation language. It uses JjEL as its expression sub-language.

3.1 Design Philosophy

JjTL targets the “sweet spot” between ATL’s complexity and manual Java code:

- **Simpler than ATL:** No two-phase execution, no `thisModule` reference, no distinction between matched/lazy/called rules. Every rule is a class mapping.
- **More structured than ETL:** Attribute mappings use a declarative binding syntax (`->`) rather than imperative assignment. Guards are explicit (**when**) rather than statement blocks.
- **JjEL instead of OCL:** Expressions use JjEL’s set-theoretic semantics, null-safe navigation, and native naming conventions, features that OCL does not provide.

3.2 Transformation Structure

A JjTL transformation declares its name, source metamodel, target metamodel, and a set of class mappings and helper functions:

```
1 transformation StateMachine2PetriNet
2
3 from StateMachineMM
```

```

4  to PetriNetMM
5
6  # Class mappings follow
7  State -> Place {
8      name -> label
9      isInitial -> tokens : true=1, false=0
10 }
11
12 # Helper functions
13 helper formatLabel(s: String) -> String {
14     s.toUpperCase()
15 }

```

3.3 Class Mappings

A class mapping specifies a correspondence between a source class and a target class:

```

SourceClass -> TargetClass [multiplicity] when { guard } {
    attributeMappings...
}

```

3.3.1 Multiplicity

Multiplicity controls how many target instances are created per source instance:

- No multiplicity: one-to-one (default).
- [*]: unbounded (used with collection sources).
- [n]: exactly n target instances per source.
- [m..n]: between m and n instances.

3.3.2 Guard Conditions

The `when` clause filters which source instances are transformed. The guard is a JjEL expression evaluated with the source instance as implicit context:

```

Class -> Table when { not isAbstract and attributes.isNotEmpty } {
    ...
}

```

The implicit context means property access is unqualified: `isAbstract` refers to `source.isAbstract`. This reduces verbosity compared to ATL (where the guard must use the bound variable: `c.isAbstract`) and QVT-O (where `self.isAbstract` is required).

3.4 Attribute Mappings

3.4.1 Direct Mapping

The simplest form copies a value from source to target:

```

name -> tableName

```

3.4.2 Value Mapping (Enum Conversion)

Discrete value conversions use the `:` separator followed by comma-separated pairs:

```
isInitial -> tokens : true=1, false=0
status -> code : "active"=1, "inactive"=0, "deleted"=-1
```

Unmapped values pass through unchanged (fallback identity).

3.4.3 Expression Mapping

Arbitrary JjEL expressions compute the target value:

```
name -> tableName : name.snakeCase()
-> columnDefs : forall a in attributes: a.name + " " + a.type.toUpper()
```

3.4.4 Object Creation

Nested target objects are created inline:

```
-> inputArcs {
  -> Arc {
    place -> source
    weight -> 1
  }
}
```

3.5 Helper Functions

Helpers are named, reusable JjEL expressions:

```
helper formatLabel(name: String, prefix: String) -> String {
  prefix + "_" + name.snakeCase()
}
```

Helpers are registered in the evaluation context and callable from any expression.

3.6 Trace Model

JjTL automatically records a trace model during execution, linking each source element to its target element(s) with binding-level detail:

- **Rule-level trace:** which class mapping produced which target instance.
- **Binding-level trace:** for each attribute mapping, the source value, target value, expression used, and whether the mapping is *invertible*.

3.6.1 Invertibility Analysis

JjTL automatically determines whether each attribute mapping can be reversed:

- Direct mappings (`a -> b`): always invertible.
- Value mappings (`true=1, false=0`): invertible if and only if the mapping is injective (no duplicate target values).

- Expression mappings: invertible only for simple property references; function calls and arithmetic are marked non-invertible.

3.7 Interactive Features

JjTL supports user interaction during transformation execution via four built-in statements:

Construct	Behavior	Returns
<code>alert(msg, type)</code>	Blocking modal	–
<code>notify(msg, duration)</code>	Non-blocking toast	–
<code>prompt(msg, default)</code>	Text input dialog	String
<code>input(msg, type, options)</code>	Typed input dialog	Varies

These enable semi-automated transformations where certain decisions require human input, a feature absent from ATL, ETL, QVT-R, and QVT-O.

3.8 Execution Model

The JjTL executor follows a simple pipeline:

1. **Initialize:** bind source model elements in the evaluation context; register helpers.
2. **Match:** for each class mapping, find all source instances of the matching type.
3. **Guard:** filter instances through the **when** condition.
4. **Create:** instantiate target elements according to multiplicity.
5. **Bind:** evaluate attribute mappings and assign values.
6. **Trace:** record the trace link with invertibility information.

Unlike ATL’s two-phase execution (where all bindings are computed before any imperative blocks run), JjTL processes each rule atomically: match, create, bind. This simplifies reasoning about execution order.

4 Comparative Analysis

We now compare JjTL/JjEL with four established transformation languages, examining both syntactic design and underlying philosophy.

4.1 ATL (Atlas Transformation Language)

4.1.1 Overview

ATL is a hybrid declarative/imperative M2M transformation language developed at INRIA as part of the ATLAS group. It uses OCL as its expression sub-language and has been the most widely used academic transformation language since the mid-2000s.

4.1.2 Structural Comparison

Aspect	ATL	JjTL
Header	<code>module Name; create Out from In</code>	<code>transformation Name \n from X \n to Y</code>
Rule type	Matched / Lazy / Called / Entrypoint	Single type: class mapping
Binding	<code>targetAttr <- expr</code>	<code>sourceAttr -> targetAttr</code>
Guard	OCL expression in <code>from</code> clause	<code>when</code> block with JjEL
Trace resolution	Implicit via matched rules	Implicit (like ATL) with per-binding invertibility
Imperative block	<code>do { ... } (post-binding)</code>	<code>do in forall (per-element)</code>
Expression language	OCL	JjEL

4.1.3 Key Design Differences

Binding direction. ATL uses `<-` (“target receives from expression”), while JjTL uses `->` (“source maps to target”). The JjTL direction mirrors the natural reading of transformation rules: the *source* drives the transformation. ATL’s direction mirrors assignment (`x := expr`).

Rule taxonomy. ATL distinguishes four rule types (matched, lazy, called, entrypoint), each with different execution semantics. JjTL has a single rule type with optional multiplicity and guard, reducing the conceptual surface.

Two-phase execution. ATL’s matched rules execute in two phases: first, all target elements are created and bindings computed (declarative phase); then, `do` blocks run (imperative phase). This design enables inter-rule references during binding but makes execution order non-obvious. JjTL processes each mapping atomically: `match`, `create`, `bind`.

Expression language. ATL uses OCL, which lacks null-safe navigation, naming convention methods, and set comprehension via `forall`. OCL collection operations (`->select()`, `->collect()`) require lambda syntax (`e | expr`) and explicit collection type management (`Set`, `Sequence`, `Bag`, `OrderedSet`).

4.2 EOL / ETL (Epsilon Platform)

4.2.1 Overview

EOL is the imperative base language of the Epsilon platform; ETL extends it for M2M transformations. EOL takes a pragmatic approach, designed to be familiar to Java/JavaScript developers.

4.2.2 Structural Comparison

Aspect	ETL	JjTL
Rule syntax	<code>rule Name transform s:T to t:T { ... }</code>	<code>S -> T { ... }</code>
Binding	Imperative assignment: <code>t.x = s.y</code>	Declarative: <code>y -> x</code>
Guard	<code>guard: expr or guard { block }</code>	<code>when { expr }</code>
Trace resolution	<code>::=</code> operator / <code>equivalent()</code>	Implicit with per-binding invertibility
Rule modifiers	<code>@abstract</code> , <code>@lazy</code> , <code>@primary</code> , <code>@greedy</code>	Multiplicity (<code>[*]</code>)
Inheritance	<code>extends parentRule</code>	Not supported
Control flow	<code>for</code> , <code>if/else</code> , <code>while</code> , <code>switch</code>	<code>forall</code> , <code>if/then/else</code>

4.2.3 Key Design Differences

Declarative vs. imperative bindings. ETL attribute mappings are imperative assignments (`t.name = s.name.toUpperCase()`). JjTL attribute mappings are declarative bindings (`name -> tableName : name.toUpperCase()`). The declarative approach enables automatic invertibility analysis, which is impossible with imperative assignments.

Verbosity. ETL requires explicit variable declarations, `new` for object creation, and method call syntax for collection operations. JjTL infers targets from the rule header, creates objects implicitly, and uses `forall` instead of explicit loops or lambda chains.

Rule inheritance. ETL supports `extends` for rule inheritance (abstract rules with shared logic). JjTL does not have rule inheritance, favoring flat, self-contained rules. This is a deliberate simplicity trade-off.

4.3 QVT Relations (QVT-R)

4.3.1 Overview

QVT-R is the declarative relational sub-language of the OMG QVT standard. Its distinguishing features are bidirectional specification and pattern-based relation declarations.

4.3.2 Structural Comparison

Aspect	QVT-R	JjTL
Paradigm	Fully declarative, bidirectional	Declarative, bidirectional (with unidirectional extension)
Rule structure	<code>relation</code> with domain patterns	Class mapping with attribute bindings
Directionality	<code>checkonly</code> / <code>enforce</code> per domain	Bidirectional core; unidirectional extension
Precondition	<code>when { ... }</code> (relation calls)	<code>when { JjEL expr }</code>
Postcondition	<code>where { ... }</code> (relation calls)	Not explicit (sequential)
Expression language	OCL	JjEL
Variable binding	Implicit via pattern variables	Explicit via source property names

4.3.3 Key Design Differences

Bidirectionality. QVT-R's core promise is that the same transformation can be executed in either direction. In practice, this creates significant complexity: users must reason about enforcement behavior in both directions simultaneously, and non-bijective mappings (common in real transformations like class-to-table) introduce semantic ambiguities. JjTL's declarative core is also bidirectional, but takes a different approach: invertibility is tracked per-binding rather than per-transformation, giving fine-grained control over which parts of a mapping can be reversed. Additionally, JjTL provides a unidirectional extension for cases where source-to-target-only semantics are needed.

Pattern matching vs. explicit mapping. QVT-R uses pattern-based domain declarations where variables are bound across domains by name equality. This is elegant but hard to read: understanding which variables bind to what requires scanning the entire relation. JjTL uses explicit `->` mappings, making the data flow immediately visible.

Relation calls. QVT-R's `when/where` clauses invoke other relations, creating a web of inter-dependent correspondences. This compositional power is also a source of complexity: debugging requires tracing through chains of relation calls. JjTL guards are self-contained JjEL expressions.

4.4 QVT Operational (QVT-O)

4.4.1 Overview

QVT-O is the imperative sub-language of the QVT standard. It uses **mapping** operations with automatic trace management, OCL-based expressions, and a three-section body (`init/population/end`).

4.4.2 Structural Comparison

Aspect	QVT-O	JjTL
Entry point	Explicit <code>main()</code>	Implicit (all rules fire)
Mapping syntax	<code>mapping T::op() : TT</code>	<code>S -> T { ... }</code>
Implicit context	<code>self</code>	Unqualified (source properties)
Object creation	<code>object Type { ... }</code>	Nested <code>-> Type { ... }</code>
Guard	<code>when { ... }</code>	<code>when { ... }</code>
Trace resolution	Implicit via mapping invocation	Implicit with per-binding invertibility
Assignment	<code>:=</code>	<code>-></code>
Expression language	OCL	JjEL

4.4.3 Key Design Differences

Entry point. QVT-O requires an explicit `main()` that orchestrates the transformation by navigating the source model and invoking mappings. JjTL fires all class mappings automatically for matching source instances (pattern-matched, like ATL). This reduces boilerplate but limits control over execution order.

Self vs. implicit context. QVT-O uses `self` to refer to the source element within a mapping. JjTL uses an implicit context: properties of the source element are accessed without qualification. This is a significant verbosity reduction but requires clear scoping rules.

4.5 Feature Comparison Matrix

Feature	ATL	ETL	QVT-R	QVT-O	JjTL
Declarative rules	✓	–	✓	–	✓
Imperative blocks	✓	✓	–	✓	Limited ^a
Bidirectional	–	–	✓	–	✓ ^e
Automatic trace	✓	✓	✓	✓	✓
Invertibility tracking	–	–	–	–	✓
Guard conditions	✓	✓	✓ ^b	✓	✓
Multiplicity	–	–	–	–	✓
Null-safe navigation	–	✓ ^c	–	–	✓
Naming conventions	–	–	–	–	✓
Interactive (user input)	–	–	–	–	✓
Set comprehension (<code>forall</code>)	–	–	–	–	✓
Rule inheritance	–	✓	–	✓ ^d	–
Expression language	OCL	EOL	OCL	OCL	JjEL

^a Via `forall...do` in expressions. ^b Via `when/where` clauses. ^c Since Epsilon 2.1. ^d Via mapping inheritance. ^e Declarative core is bidirectional; also has a unidirectional extension.

4.6 Expression Language Comparison

A critical differentiator is the expression sub-language. We compare equivalent expressions across OCL (used by ATL, QVT-R, QVT-O), EOL (used by ETL), and JjEL (used by JjTL):

Task	OCL	EOL	JjEL
Public attr. names	<code>self.attr->select(a a.isPublic)->collect(a a.name)</code>	<code>self.attr->select(a a.isPublic).collect(a a.name)</code>	<code>self.attr->select(a a.isPublic).collect(a a.name)</code> attr such that a.isPublic: a.name
Exists string attr	<code>self.attr->exists(a a.type.name==String)</code>	<code>self.attr->exists(a a.type.name==String?)</code>	<code>self.attr->exists(a a.type.name==String?)</code> a.type == "String"
Abstract $\Rightarrow$ has children	<code>self.isAbstract implies self.subClasses->notEmpty()</code>	<code>self.isAbstract implies self.subClasses.notEmpty()</code>	<code>isAbstract implies subClasses.isNotEmpty</code>
Null-safe name	<code>if self.parent.oclIsUndefined() then 'none' else self.parent.name endif</code>	<code>self.parent?.name ??: "none"</code>	<code>parent?.name ?? "none"</code>
To snake_case	(no native support)	(no native support)	<code>name.snakeCase()</code>

Key observations:

1. JjEL's `forall` replaces the `->select()->collect()` chain with a single construct.
2. OCL's null handling requires verbose `oclIsUndefined()` checks.
3. Naming convention methods (`snakeCase`, `camelCase`) are unique to JjEL.
4. JjEL uses implicit context (no `self`), reducing token count.

5 Transformation Examples

We present three transformations in parallel across ATL, ETL, and JjTL, each demonstrating different language features.

5.1 Example 1: UML Class to Relational Table

This classic transformation maps UML classes to relational tables: each non-abstract class becomes a table, each single-valued attribute becomes a column, and each multi-valued attribute becomes a junction table.

5.1.1 ATL

Listing 1: Class2Relational in ATL

```

1 module Class2Relational;
2 create OUT : Relational from IN : Class;
3
4 helper context Class!Attribute def: isSingleValued() : Boolean =
5     not self.multiValued;
6
7 rule Class2Table {
8     from
9         c : Class!Class (
10             not c.isAbstract
11         )
12     to
13         t : Relational!Table (
14             name <- c.name,
15             col <- c.attr->select(a | a.isSingleValued())
16                 ->collect(a | thisModule.Attr2Col(a))
17                 ->union(Sequence{key}),
18             key <- Set{key}
19         ),
20         key : Relational!Column (
21             name <- 'Id'
22         )
23 }
24
25 lazy rule Attr2Col {
26     from
27         a : Class!Attribute
28     to
29         c : Relational!Column (
30             name <- a.name,
31             type <- a.type.name
32         )
33 }
34
35 rule MultiAttr2Table {
36     from
37         a : Class!Attribute (
38             a.multiValued
39         )
40     to
41         t : Relational!Table (
42             name <- a.owner.name + '_' + a.name,
43             col <- Sequence{id, val}
44         ),
45         id : Relational!Column (
46             name <- 'Id'
47         ),
48         val : Relational!Column (
49             name <- a.name,

```

```

50         type <- a.type.name
51     )
52 }

```

5.1.2 ETL

Listing 2: Class2Relational in ETL

```

1  rule Class2Table
2      transform c : Class!Class
3      to t : Relational!Table {
4
5          guard : not c.isAbstract
6
7          t.name = c.name;
8
9          // Primary key
10         var pk = new Relational!Column;
11         pk.name = "Id";
12         pk.type = "INTEGER";
13         t.col.add(pk);
14
15         // Single-valued attributes
16         for (a in c.attr.select(a | not a.multiValued)) {
17             var col = new Relational!Column;
18             col.name = a.name;
19             col.type = a.type.name;
20             t.col.add(col);
21         }
22     }
23
24 rule MultiAttr2Table
25     transform a : Class!Attribute
26     to t : Relational!Table {
27
28         guard : a.multiValued
29
30         t.name = a.owner.name + "_" + a.name;
31
32         var id = new Relational!Column;
33         id.name = "Id";
34         t.col.add(id);
35
36         var val = new Relational!Column;
37         val.name = a.name;
38         val.type = a.type.name;
39         t.col.add(val);
40     }

```

5.1.3 JjTL

Listing 3: Class2Relational in JjTL

```

1  transformation Class2Relational
2
3  from ClassMM
4  to RelationalMM

```

```

5
6 # Non-abstract classes become tables
7 Class -> Table when { not isAbstract } {
8     name -> name : name.snakeCase()
9
10    # Primary key column
11    -> col {
12        -> Column {
13            -> name : "id"
14            -> type : "INTEGER"
15        }
16    }
17
18    # Single-valued attributes become columns
19    -> col {
20        forall a in attributes such that not a.multiValued
21        -> Column {
22            a.name -> name : a.name.snakeCase()
23            a.type.name -> type
24        }
25    }
26 }
27
28 # Multi-valued attributes become junction tables
29 Attribute -> Table when { multiValued } {
30     -> name : owner.name.snakeCase() + "_" + name.snakeCase()
31
32     -> col {
33         -> Column {
34             -> name : "id"
35         }
36     }
37     -> col {
38         -> Column {
39             name -> name : name.snakeCase()
40             type.name -> type
41         }
42     }
43 }

```

5.2 Example 2: State Machine to Petri Net

This transformation converts a state machine (states and transitions) into a Petri net (places, transitions, and arcs).

5.2.1 ATL

Listing 4: StateMachine2PetriNet in ATL

```

1 module SM2PN;
2 create OUT : PetriNet from IN : StateMachine;
3
4 rule State2Place {
5     from
6         s : StateMachine!State
7     to
8         p : PetriNet!Place (

```

```

9         name    <- s.name,
10         tokens <- if s.isInitial then 1 else 0 endif
11     )
12 }
13
14 rule Transition2Transition {
15     from
16     t : StateMachine!Transition
17     to
18     pt : PetriNet!Transition (
19         name <- t.label
20     ),
21     inArc : PetriNet!Arc (
22         source    <- t.source, -- resolved via trace
23         target    <- pt,
24         weight    <- 1
25     ),
26     outArc : PetriNet!Arc (
27         source    <- pt,
28         target    <- t.target, -- resolved via trace
29         weight    <- 1
30     )
31 }

```

5.2.2 ETL

Listing 5: StateMachine2PetriNet in ETL

```

1 rule State2Place
2     transform s : SM!State
3     to p : PN!Place {
4
5         p.name    = s.name;
6         p.tokens = if (s.isInitial) 1 else 0;
7     }
8
9 rule Transition2Transition
10     transform t : SM!Transition
11     to pt : PN!Transition {
12
13         pt.name = t.label;
14
15         // Input arc: from source place to this transition
16         var inArc = new PN!Arc;
17         inArc.source = t.source.equivalent();
18         inArc.target = pt;
19         inArc.weight = 1;
20
21         // Output arc: from this transition to target place
22         var outArc = new PN!Arc;
23         outArc.source = pt;
24         outArc.target = t.target.equivalent();
25         outArc.weight = 1;
26     }

```

5.2.3 JjTL

Listing 6: StateMachine2PetriNet in JjTL

```

1 transformation StateMachine2PetriNet
2
3 from StateMachineMM
4 to   PetriNetMM
5
6 State -> Place {
7     name -> name
8     isInitial -> tokens : true=1, false=0
9 }
10
11 Transition -> Transition [*] {
12     label -> name
13
14     # Input arc: source state -> this transition
15     -> inputArcs {
16         -> Arc {
17             source -> source
18             -> weight : 1
19         }
20     }
21
22     # Output arc: this transition -> target state
23     -> outputArcs {
24         -> Arc {
25             target -> target
26             -> weight : 1
27         }
28     }
29 }

```

Comparison notes.

- ATL and JjTL both use implicit trace resolution: when a source element (e.g., a **State**) is assigned to a target property that expects the corresponding target type (e.g., a **Place**), the engine automatically resolves the trace link. This keeps the mapping syntax clean and declarative.
- ETL requires explicit `.equivalent()` calls to resolve trace links.
- The value mapping `true=1, false=0` in JjTL replaces ATL's verbose `if...then...else...endif`.

5.3 Example 3: UML to Java (Structural)

This example maps UML classes to Java class definitions, demonstrating naming conventions, inheritance, and collection processing.

5.3.1 ATL

Listing 7: UML2Java structural mapping in ATL

```

1 module UML2Java;
2 create OUT : Java from IN : UML;
3
4 helper context UML!Class def: javaName() : String =
5     self.name;
6     -- Note: no native camelCase/PascalCase in OCL

```

```

7
8 helper context UML!Attribute def: getterName() : String =
9     'get' + self.name;  -- cannot capitalize first letter natively
10
11 rule Class2JavaClass {
12     from
13         c : UML!Class
14     to
15         j : Java!JavaClass (
16             name      <- c.javaName(),
17             isAbstract <- c.isAbstract,
18             superClass <- c.superClass,  -- trace-resolved
19             fields     <- c.attributes->select(a |
20                 not a.isDerived),
21             methods    <- c.attributes->select(a |
22                 not a.isDerived)
23                 ->collect(a | thisModule.Attr2Getter(a))
24         )
25 }
26
27 lazy rule Attr2Getter {
28     from
29         a : UML!Attribute
30     to
31         m : Java!Method (
32             name      <- a.getterName(),
33             returnType <- a.type,
34             body       <- 'return this.' + a.name + ';'
35         )
36 }

```

5.3.2 ETL

Listing 8: UML2Java structural mapping in ETL

```

1 rule Class2JavaClass
2     transform c : UML!Class
3     to j : Java!JavaClass {
4
5         j.name      = c.name;
6         j.isAbstract = c.isAbstract;
7         j.superClass ::= c.superClass;
8
9         for (a in c.attributes.select(a | not a.isDerived)) {
10             var field = new Java!Field;
11             field.name = a.name;
12             field.type ::= a.type;
13             j.fields.add(field);
14
15             var getter = new Java!Method;
16             getter.name = "get" + a.name.firstToUpperCase();
17             getter.returnType ::= a.type;
18             getter.body = "return this." + a.name + ";";
19             j.methods.add(getter);
20         }
21 }

```

5.3.3 JjTL

Listing 9: UML2Java structural mapping in JjTL

```

1 transformation UML2Java
2
3 from UML_MM
4 to   JavaMM
5
6 Class -> JavaClass {
7     name -> name : name.pascalCase()
8     isAbstract -> isAbstract
9     superClass -> superClass
10
11     # Fields from non-derived attributes
12     -> fields {
13         forall a in attributes such that not a.isDerived
14         -> Field {
15             a.name -> name : a.name.camelCase()
16             a.type -> type
17         }
18     }
19
20     # Getter methods
21     -> methods {
22         forall a in attributes such that not a.isDerived
23         -> Method {
24             -> name : "get" + a.name.pascalCase()
25             a.type -> returnType
26             -> body : "return this." + a.name.camelCase() + ";"
27         }
28     }
29 }

```

Comparison notes.

- JjTL's native `pascalCase()` and `camelCase()` methods eliminate the need for manual string manipulation helpers that ATL and ETL require.
- ATL needs a lazy rule (`Attr2Getter`) invoked via `thisModule.Attr2Getter(a)`; JjTL uses inline object creation.
- ETL's imperative loop with explicit `new` and `.add()` is more verbose than JjTL's declarative `forall` with nested object creation.
- OCL's string handling is the weakest: there is no native method to capitalize the first letter, convert to `camelCase`, etc.

5.4 Example 4: Validation Invariants

This example demonstrates JjEL's expressiveness for model validation, compared to OCL.

Invariant	OCL	JjEL
All classes have ≥ 1 attribute	<code>Class.allInstances()->forall(c c.attributes->notEmpty())</code>	<code>forall c in classes: c.attributes.isNotEmpty</code>
Abstract $\Rightarrow$ has sub-classes	<code>self.isAbstract implies self.subClasses->notEmpty()</code>	<code>isAbstract implies subClasses.isNotEmpty</code>
Unique class names	<code>Class.allInstances()->isUnique(c c.name)</code>	<code>(forall c in classes: c.name).distinct().size == classes.size</code>
No attribute with null type	<code>self.attributes->forall(a not a.type.ocIsUndefined())</code>	<code>forall a in attributes: a.type != null</code>
Public attributes $\Rightarrow$ typed	<code>self.attributes->select(a a.isPublic)->forall(a not a.type.ocIsUndefined())</code>	<code>forall a in attributes such that a.isPublic: a.type != null</code>

In JjEL, the same `forall` construct serves for both selection and projection, while OCL requires different operations (`->forall()` for assertion, `->select()` for filtering, `->collect()` for mapping). Note also that in JjEL validation, `forall` with `:` returns a set of boolean values; the invariant holds if all values are `true`. An explicit `.all(x => x)` or wrapping with `forall ... such that` can make the boolean intent more explicit.

5.5 Example 5: Guard Conditions

Guards in transformation rules demonstrate how expression languages handle complex selection criteria.

Guard	ATL (OCL)	ETL (EOL)	JjTL (JjEL)
Non-abstract with attributes	<code>not c.isAbstract and c.attr->notEmpty()</code>	<code>not c.isAbstract and c.attr.notEmpty()</code>	<code>not isAbstract and attributes.isNotEmpty</code>
Has a string attribute	<code>c.attr->exists(a a.type.name = 'String')</code>	<code>c.attr.exists(a a.type.name = 'String')</code>	<code>exists a in attributes: a.type == "String"</code>
All references are navigable	<code>c.refs->forall(r r.isNavigable)</code>	<code>c.refs.forAll(r r.isNavigable)</code>	<code>forall r in references: r.isNavigable</code>

6 Discussion

6.1 Strengths of the JjTL/JjEL Approach

1. **Conceptual unification via forall:** By giving `forall` set-theoretic semantics, JjEL replaces three distinct concepts (filter, map, logical quantification) with one. This is a genuine reduction in cognitive load, not merely syntactic sugar.
2. **Domain-native operations:** Built-in naming convention methods (`camelCase`, `snakeCase`, etc.) and null-safe navigation address the most common pain points in model transformations, which no other transformation language handles natively.
3. **Invertibility tracking:** JjTL's automatic classification of attribute mappings as invertible or non-invertible is unique among the compared languages and enables downstream applications like bidirectional synchronization and transformation debugging.
4. **Interactive transformations:** The `prompt/input` mechanism supports semi-automated scenarios that other languages cannot express.

5. **Single execution model:** Unlike ATL’s two-phase matched rules or QVT-R’s complex enforcement semantics, JjTL’s execution model is straightforward: match, create, bind.

6.2 Limitations and Trade-offs

1. **Limited imperative support:** JjTL’s `do` blocks provide basic imperative capability, but lack the full expressiveness of ATL’s called rules or QVT-O’s mapping operations for complex procedural logic.
2. **No rule inheritance:** Unlike ETL’s `extends` and QVT-O’s mapping inheritance, JjTL rules are flat. Complex transformations with shared logic across rules must use helper functions.
3. **No explicit entry point:** The automatic “fire all matching rules” execution model (like ATL) limits control over rule ordering. QVT-O’s explicit `main()` provides more control.
4. **Maturity:** ATL and ETL have decades of academic use, extensive example repositories, and battle-tested implementations. JjTL/JjEL are new and will require the same maturation process.

6.3 Open Design Points

Several aspects of JjEL are under active review:

1. **if/then/else syntax:** The conditional has a procedural flavor. Alternatives like `when/then/otherwise` have been considered for their more declarative reading.
2. **String interpolation:** The lexer supports `${...}` syntax but the parser does not yet produce interpolated string nodes. This is planned as a future enhancement.
3. **Closure method:** A generic `closure()` method for transitive navigation would replace hardcoded properties like `allSuperclasses` and `allSubclasses`.
4. **forall nesting:** The readability and semantics of nested `forall` expressions need further validation.

7 Conclusion

This document has presented JjTL and JjEL as a pair of complementary domain-specific languages for Model-Driven Engineering. The key contribution is JjEL’s set-theoretic approach to expressions: by reinterpreting `forall` as a set comprehension rather than a logical assertion, we achieve a significant reduction in cognitive load while maintaining the declarative character valued in the MDE community.

The comparative analysis with ATL, EOL/ETL, QVT-R, and QVT-O reveals that each language occupies a distinct design point:

- ATL combines declarative rules with OCL expressions but suffers from two-phase execution complexity and limited tooling.
- ETL prioritizes developer familiarity via an imperative style but sacrifices the declarative invertibility that enables advanced transformation analysis.
- QVT-R achieves full declarativity and bidirectionality at the cost of practical usability.
- QVT-O provides a pragmatic imperative approach with OCL’s verbosity.

JjTL/JjEL targets the space between ATL’s declarative elegance and ETL’s practical simplicity, aiming for a language that is *declarative enough* for transformation analysis (invertibility, traceability) while being *intuitive enough* for adoption by domain experts who may not have a formal logic background.

A JjEL Grammar (Complete)

Listing 10: JjEL formal grammar in EBNF

```

1 expression      = forallExpr | existsExpr | withExpr | ifThenElse
2
3 forallExpr      = 'forall' IDENT 'in' expression
4                  ('such' 'that' expression)?
5                  (':' expression)?
6                  ('do' expression)?
7
8 existsExpr      = 'exists' IDENT 'in' expression
9                  (':' | 'such' 'that') expression
10
11 withExpr       = 'with' expression 'do' expression
12
13 ifThenElse     = 'if' expression 'then' expression ('else' expression)?
14                  | nullCoalesce
15
16 nullCoalesce   = implies ('??' implies)*
17
18 implies        = logicalOr ('implies' logicalOr)?
19
20 logicalOr      = logicalAnd ('or' logicalAnd)*
21
22 logicalAnd     = equality ('and' equality)*
23
24 equality        = comparison (('==' | '!=') comparison)*
25
26 comparison     = isType (('<' | '>' | '<=' | '>=') isType)*
27
28 isType         = addition ('is' IDENT)?
29
30 addition       = multiplication (('+' | '-') multiplication)*
31
32 multiplication = unary (('*' | '/' | '%') unary)*
33
34 unary          = 'not' unary
35                  | '-' unary
36                  | postfix
37
38 postfix        = primary (
39                  ' ' IDENT '(' argList ')')?
40                  | '?. ' IDENT '(' argList ')')?
41                  | '[' expression ']'
42                  )*
43
44 primary        = literal | IDENT | lambda | arrayLiteral
45                  | '(' expression ')'
46
47 lambda         = IDENT '=>' expression
48                  | '(' IDENT (',' IDENT)* ') '=>' expression

```

```

49
50 arrayLiteral    = '[' (expression (',' expression)*)? ']'
51
52 argList         = (expression (',' expression)*)?
53
54 literal         = STRING | NUMBER | 'true' | 'false' | 'null'

```

B JjEL Built-in Methods (Complete)

B.1 String Methods (33)

Method	Parameters	Description
toUpperCase()	–	Uppercase
toLowerCase()	–	Lowercase
capitalize()	–	First letter uppercase
uncapitalize()	–	First letter lowercase
camelCase()	–	"my name" → "myName"
pascalCase()	–	"my name" → "MyName"
snakeCase()	–	"myName" → "my_name"
kebabCase()	–	"myName" → "my-name"
trim()	–	Remove surrounding whitespace
trimStart()	–	Remove leading whitespace
trimEnd()	–	Remove trailing whitespace
padStart(n, ch)	length, char	Pad start to length
padEnd(n, ch)	length, char	Pad end to length
contains(s)	substring	Contains check
startsWith(s)	prefix	Prefix check
endsWith(s)	suffix	Suffix check
indexOf(s)	substring	First occurrence index
lastIndexOf(s)	substring	Last occurrence index
matches(r)	regex	Regex match
replace(o, n)	old, new	Replace first
replaceAll(o, n)	old, new	Replace all
substring(s, e?)	start, end?	Extract substring
slice(s, e?)	start, end?	Slice (negative indices)
split(sep)	separator	Split to array
repeat(n)	count	Repeat string
reverse()	–	Reverse string
length	–	Character count
isEmpty	–	Length == 0
isNotEmpty	–	Length > 0
isBlank	–	Empty or whitespace only
isNotBlank	–	Not blank
charAt(i)	index	Character at position
toNumber()	–	Parse as number
toInt()	–	Parse as integer

B.2 Collection Methods (28)

After `forall` replaces `filter()` and `map()`, the remaining collection methods are:

Method	Parameters	Description
<code>sum()</code>	–	Numeric sum
<code>avg()</code>	–	Numeric average
<code>min()</code>	–	Minimum value
<code>max()</code>	–	Maximum value
<code>count()</code>	–	Element count
<code>first</code>	–	First element
<code>last</code>	–	Last element
<code>at(i)</code>	index	Element at index
<code>indexOf(x)</code>	item	Index of element
<code>size</code>	–	Collection size
<code>length</code>	–	Alias for size
<code>isEmpty</code>	–	Size == 0
<code>isNotEmpty</code>	–	Size > 0
<code>contains(x)</code>	item	Membership test
<code>distinct()</code>	–	Remove duplicates
<code>distinctBy(fn)</code>	lambda	Remove duplicates by key
<code>sortBy(fn)</code>	lambda	Sort ascending
<code>sortByDesc(fn)</code>	lambda	Sort descending
<code>reverse()</code>	–	Reverse order
<code>flatten()</code>	–	Flatten nested arrays
<code>flatMap(fn)</code>	lambda	Map + flatten
<code>groupBy(fn)</code>	lambda	Group by key
<code>take(n)</code>	count	First n elements
<code>skip(n)</code>	count	Skip first n
<code>takeWhile(fn)</code>	predicate	Take while true
<code>skipWhile(fn)</code>	predicate	Skip while true
<code>join(sep)</code>	separator	Join to string
<code>all(fn)</code>	predicate	All satisfy?
<code>any(fn)</code>	predicate	Any satisfies?
<code>none(fn)</code>	predicate	None satisfy?

C JjTL Token Types

Category	Token	Example
Keywords	TRANSFORMATION	transformation
	FROM	from
	TO	to
	WHEN	when
	HELPER	helper
Interactive	ALERT	alert
	NOTIFY	notify
	PROMPT	prompt
	INPUT	input
Literals	STRING	"hello"
	NUMBER	42, 3.14
	BOOLEAN	true, false
	IDENTIFIER	name, State
Operators	ARROW	->
	DOT	.
	QUESTION_DOT	?.